Joint Radar Emitter and Mode Clustering Using Transformer Encoder Architecture

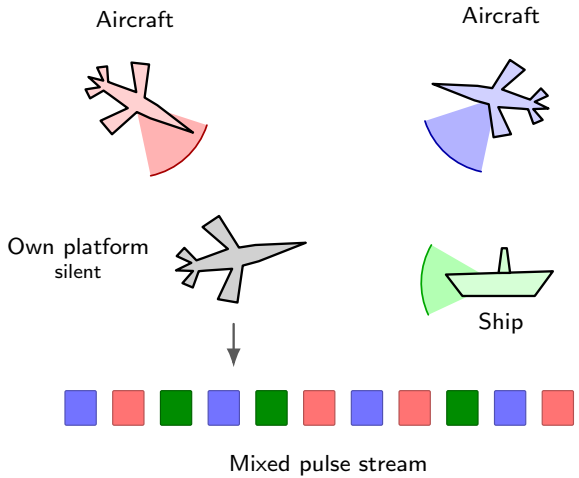
Markus Johnson Swegmark

Degree project in Machine Learning, 30 credits
School of Electrical Engineering and Computer Science
KTH Royal Institute of Technology

Supervisors: Alexander Karlsson, Elin Ohlman (Saab)
Saikat Chatterjee (KTH)
Examiner: Magnus Jansson Host: Saab AB



The situation



- The own platform listens and does not transmit.
- Several emitters radiate at once.
- One mixed stream of pulses arrives.

Terminology

Electronic Support Measures (ESM)

The listener. It detects, intercepts, and analyzes electromagnetic signals without transmitting.

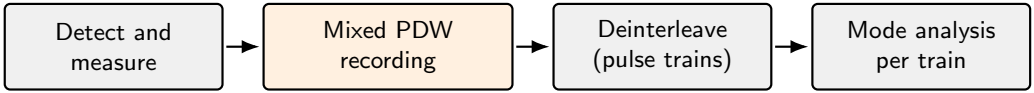
Pulse Descriptor Word (PDW)

One measured pulse, stored as a discrete record of its parameters.

- Time of arrival (TOA)
- Carrier frequency
- Pulse width (PW)
- Amplitude
- Angle of arrival (AOA)
  Direction the pulse came from.

Problem

A typical ESM processing chain:

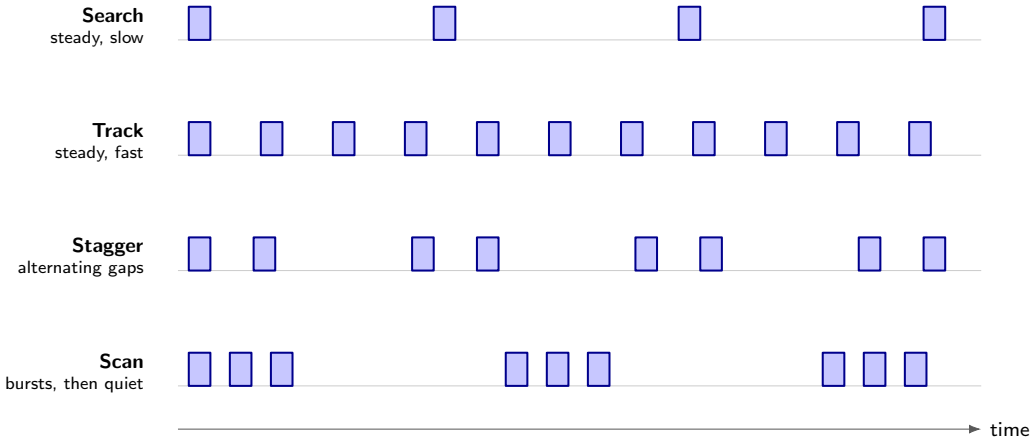


- Mode classification runs on deinterleaved streams, so it sees less of the mixture. Pulses dropped by a poor deinterleaving never reach it.
- A classifier does not handle new emitters naturally. Clustering is the broader alternative.

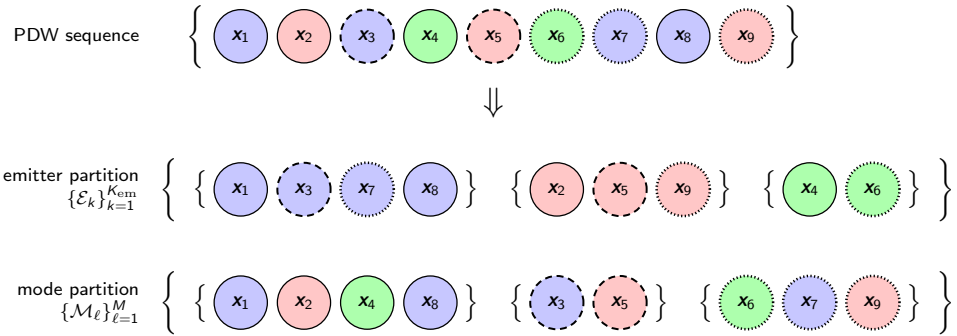
Hypothesis

Joint deinterleaving and mode clustering can do better: one model for both.

Operating modes



One mixed stream, two partitions



Emitter · deinterleaving

- Physical source
- Cluster within the recording
- What classical deinterleaving recovers

Mode · operating schedule

- Operating mode
- One mode can sit on several emitters; one emitter can switch modes
- Inference clusters. The catalogue is used in training, not at inference

Research questions

RQ1. Given a bounded window of PDWs, can a transformer encoder deinterleave pulses and recover operating modes by clustering?

RQ2. Does a joint objective that couples deinterleaving with mode clustering outperform a comparable transformer trained for deinterleaving alone?

RQ3. Do a pairwise physical attention bias and rotary positional encoding on pulse coordinates improve clustering relative to standard multi-head attention?

Data

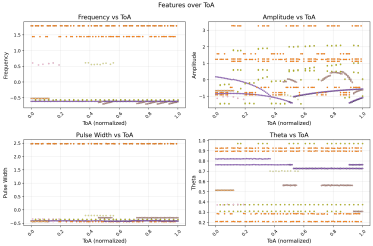
- Generated with Saab's scenario simulator.
- 1950 scenarios: 1461 train, 245 validation, and 244 test. Catalogue of 19 modes.
- Scenarios are cut into windows of length $L = 2000$ so they fit the transformer.

Split	Scenarios	Pulses	Windows	Emitters	Modes
Train	1461	13.8×10^6	56 029	3.15 ± 1.14	6.16 ± 2.95
Validation	245	2.25×10^6	1 107	3.16 ± 1.21	6.18 ± 2.95
Test	244	2.36×10^6	1 169	3.17 ± 1.20	6.18 ± 3.02

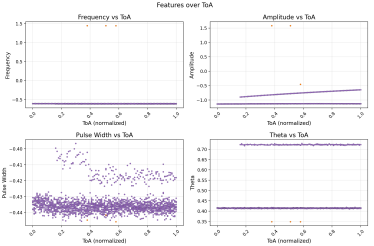
Emitter and mode columns are mean $\pm$ std distinct labels per window. Training windows overlap; validation and test windows do not.

Examples of scenarios

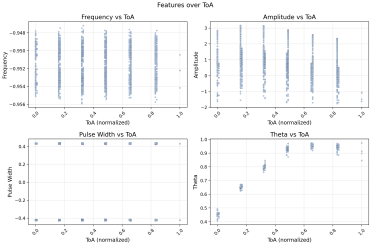
Crowded



Imbalance



Sparse



Model architecture

Transformer encoder

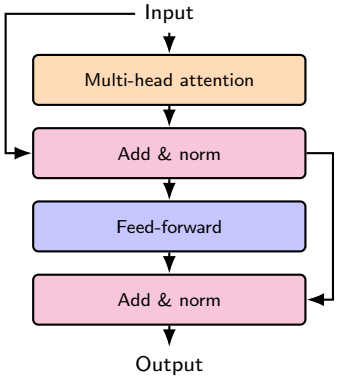
- Input: L (sequence length) $\times$ F (feature dimension)
- Output embeddings: $L \times D$, with $D = 256$

Strengths

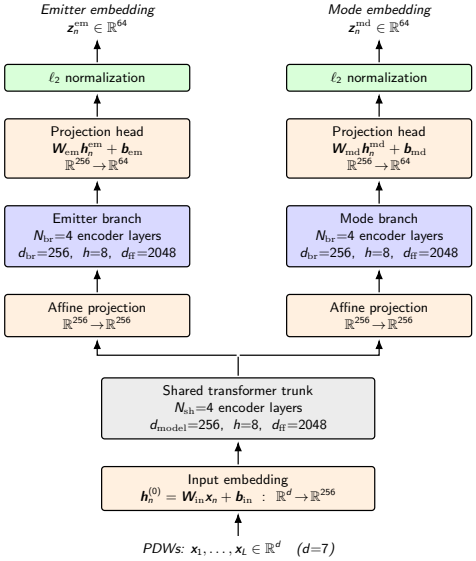
- A strong sequence model
- Returns an embedding per pulse
- Widely studied

Weaknesses

- $O(L^2)$ memory and compute
- Long streams must be windowed
- Order is not built in

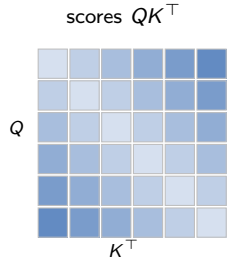


Shared trunk, two heads, two positive sets



- The trunk and both branches are stacks of transformer encoder layers.
- A projection head reduces the dimension before ℓ_2 normalization.
- 16.1 million parameters.
- DBSCAN is applied to the embeddings at inference.
- Training does not run DBSCAN. The loss uses the embeddings directly.

Attention mechanism



A similarity matrix: a complete weighted graph on the L pulses.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

- Each row of $QK^\top$ scores one pulse against every other pulse in the window.
- Softmax turns that row into weights. The pulse is then a mixture of the values V.
- Nothing in this product knows the order of the pulses, or their TOA.

Two ways to put position into attention

Rotary encodings (RoPE-TOA)

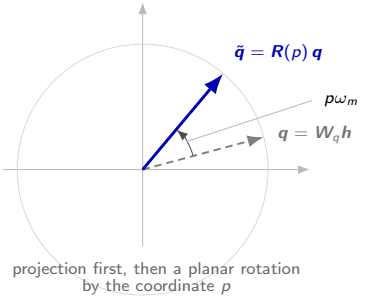
- Rotate Q and K by the window-local TOA.
- The score is a dot product, so only the enclosed angle matters.
- Only the relative rotation $R(\Delta t)$ enters the product.
- In language models the angle is the token index. Here it is TOA, because PDWs are not equally spaced.
- No extra memory, and about $O(L)$ extra compute.

Physical bias

- Build $B_{ij} = |p_i - p_j|$ from TOA and incidence.
- Scale by a learned λ and subtract from the attention logits.
- One λ per head and per stack, so heads can penalize or reward different deltas.
- $O(L^2)$ compute and memory, the same order as attention itself.
- Uncommon in language models, because of that quadratic cost.

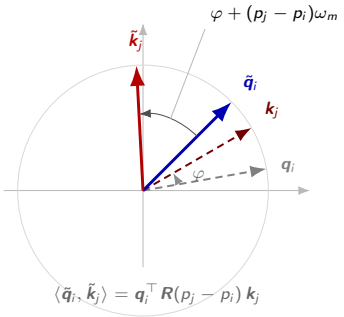
RoPE-TOA: each plane rotates at its own frequency

Apply $R(p)$ to the projected query



One head is split into 2D planes. Each plane has its own frequency.

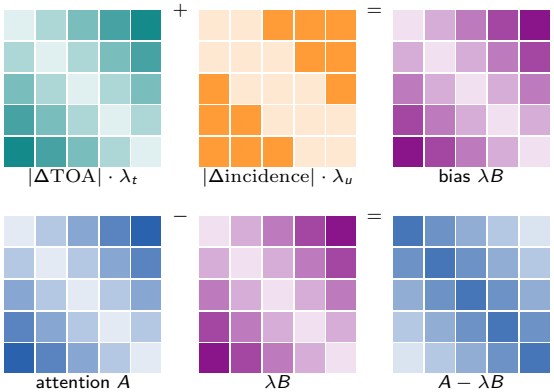
Inner product sees only $p_j - p_i$



Equal TOA leaves Q and K as they are. It does not pull them together or apart.

The score keeps only the relative gap, which twists the fast planes more.

Physical bias: learned λ on delta maps



Nearby, same-direction pulses stay bright. The maps are $L \times L$. That is the cost.

- $|\Delta\mathrm{TOA}|$ and $|\Delta\mathrm{incidence}|$ come from the inputs.
- λ_t and λ_u are learned scalars, one pair per head.
- The bias is subtracted from the vanilla logits.
- A cache can skip rebuilding B. It does not skip the quadratic subtract.

Loss function

Supervised contrastive loss. Pulses that share a label are the positives. Every other pulse in the batch is a candidate.

$$s_{ij} \coloneqq \boldsymbol{z}_i^\top \boldsymbol{z}_j \in [-1, 1]$$

$$\mathcal{L}_{\mathrm{SC}} = -\sum_{i \in \mathcal{B}^+} \frac{1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(s_{ip}/\tau)}{\sum_{a \in A(i)} \exp(s_{ia}/\tau)}$$

$A(i)$ is every pulse in the batch except i. $P(i) \subseteq A(i)$ is those that share its label. $\mathcal{B}^+$ is the pulses with at least one positive. $\tau = 0.5$.

The same expression separates into repulsion and attraction:

$$\mathcal{L}_{\mathrm{SC}} = \underbrace{\sum_{i \in \mathcal{B}^+} \log Z_i}_{\text{log-partition (repulsion)}} - \underbrace{\sum_{i \in \mathcal{B}^+} \frac{1}{|P(i)|} \sum_{p \in P(i)} \frac{s_{ip}}{\tau}}_{\text{mean positive similarity (attraction)}}, \quad Z_i = \sum_{a' \in A(i)} \exp(s_{ia'}/\tau).$$

Joint training

$$\mathcal{L}_{\text{joint}} = \mathcal{L}_{\text{em}} + \mathcal{L}_{\text{md}}.$$

- The two terms are the same loss on different positive sets.
- Emitter positives share a recording-local emitter identity, so the term trains deinterleaving.
- Mode positives share a catalogue mode, and the comparison runs across the batch.

Task-specific models keep the same two branches but train on one term only. The other branch is still computed and then discarded, which is a pure deinterleaving transformer or a pure mode transformer.

Deinterleaving: mean $\pm$ std over 1169 windows

Model	ARI	Count error
Vanilla	0.948 ± 0.176	0.25 ± 0.49
RoPE-TOA	$\mathbf{1.000 \pm 0.003}$	0.11 ± 0.34
RoPE-az	0.973 ± 0.135	0.16 ± 0.40
Bias	$\mathbf{1.000 \pm 0.002}$	$\mathbf{0.10 \pm 0.33}$
RoPE-TOA+Bias	0.851 ± 0.255	0.73 ± 0.94

Joint versus emitter-only.

Model	Joint ARI	Emitter-only ARI
Vanilla	0.948 ± 0.176	0.999 ± 0.012
RoPE-TOA	1.000 ± 0.003	0.998 ± 0.036
Bias	1.000 ± 0.002	0.840 ± 0.265

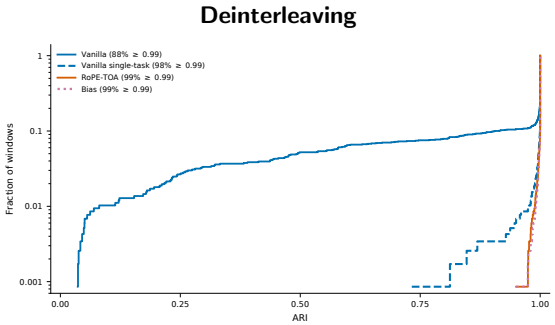
Mode clustering: mean $\pm$ std over 1169 windows

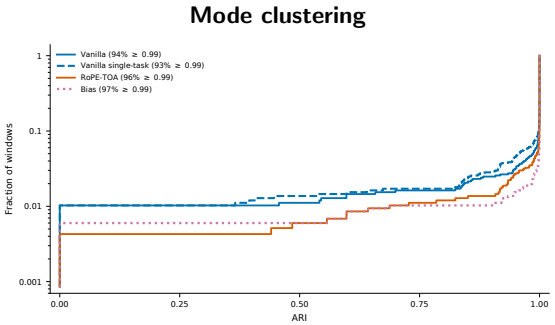
Model	ARI	Count error
Vanilla	0.985 ± 0.107	0.23 ± 0.50
RoPE-TOA	0.991 ± 0.075	0.20 ± 0.48
RoPE-az	0.983 ± 0.102	0.24 ± 0.51
Bias	$\mathbf{0.992 \pm 0.081}$	$\mathbf{0.14 \pm 0.41}$
RoPE-TOA+Bias	0.977 ± 0.112	0.32 ± 0.60

Joint versus mode-only on the Vanilla encoder.

Objective	ARI	Count error
Joint	$\mathbf{0.985 \pm 0.107}$	$\mathbf{0.23 \pm 0.50}$
Mode-only	0.983 ± 0.110	0.24 ± 0.56

Distribution of scores over windows





Conclusion

Deinterleaving

- The joint model is weaker than the single-task model when attention is left unmodified.
- With RoPE-TOA or the physical bias, the joint model is on par with single-task deinterleaving, and slightly ahead.

Mode clustering

The joint model is slightly better than the single-task model, and the two are essentially on par.

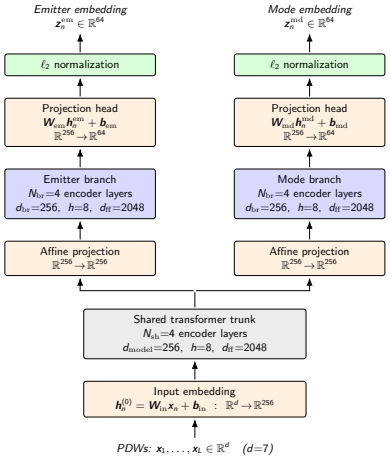
Discussion and further work

Windows

- A scenario is longer than one window.
- The window clusters still have to be stitched into a scenario-level answer.

A second head

- A mode classification head could sit on the mode embeddings, beside the clustering path.



A classification head would read the mode embeddings z_n^{md}.

Questions?